

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Discrete Structure (CO1007)

Assignment

A* Algorithms Report

Advisor(s): Trần Hồng Tài

Student(s): Lê Ân Tri 2413597

HO CHI MINH CITY, JUNE 2026



Mục lục

1	Degrees of Separation in a Social Network	5
1.1	Giải thích cách tính hàm Heuristic $h(n)$	5
1.2	Ví dụ chạy tay (Manual Example Run)	5
2	Drone Delivery in 2D Space with Three Heuristics	9
2.1	So sánh tổng quát ba Heuristic (Manhattan, Euclidean, Chebyshev)	9
2.2	Ví dụ chạy tay minh chứng sự ưu việt về hiệu suất của Manhattan	10
2.3	Ví dụ chạy tay minh chứng sự ưu việt về hiệu suất của Euclidean	13
2.4	Ví dụ chạy tay minh chứng sự ưu việt về hiệu suất của Chebyshev	16
3	Warehouse Robot Navigation with Obstacles	19
3.1	Phân tích bài toán và Mô hình hóa hệ thống	19
3.2	Ví dụ chạy tay chi tiết trên lưới tọa độ	20
3.3	Bảng tổng hợp so sánh hiệu suất định lượng của hai chế độ	23
4	Evacuation Route Planning	24
4.1	Phân tích bài toán và Mô hình hóa hệ thống	24
4.2	Ví dụ chạy tay chi tiết minh họa tiến trình ra quyết định	25
4.3	Bảng tổng hợp so sánh hiệu suất định lượng của hai chế độ	30

Danh sách hình vẽ

1.1	Sơ đồ mạng xã hội và đường đi ngắn nhất ($0 \rightarrow 1 \rightarrow 3 \rightarrow 4$)	6
2.1	Sơ đồ phân nhánh đối xứng minh họa hiệu suất vượt trội của Manhattan	10
2.2	Sơ đồ không gian mở minh họa hiệu suất vượt trội của Heuristic Euclidean	14
2.3	Sơ đồ không gian lưới 8 hướng minh họa hiệu suất vượt trội của Heuristic Chebyshev	17
3.1	Sơ đồ hệ lưới không gian nhà kho 3×3 với vật cản trung tâm ô $(1, 1)$	21
4.1	Sơ đồ phẳng hóa đồ thị tòa nhà với nút bẫy vật cản tại vị trí $N_1(1, 0)$	26

Danh sách bảng

1.1	Ma trận kề cho ví dụ mạng xã hội 5 người	5
2.1	Bảng so sánh hiệu năng định lượng của 3 Heuristic trên cấu trúc nhánh đối xứng	13



2.2	Bảng so sánh hiệu năng định lượng của 3 Heuristic trong môi trường bay tự do	16
2.3	Bảng so sánh hiệu năng định lượng của 3 Heuristic trong môi trường lưới Chebyshev	19
3.1	Bảng so sánh định lượng hiệu năng điều hướng của robot trong môi trường nhà kho	23
4.1	Bảng so sánh hiệu suất định lượng giữa Mode 1 và Mode 2	30

1 Degrees of Separation in a Social Network

1.1 Giải thích cách tính hàm Heuristic $h(n)$

Theo yêu cầu của đề bài, $h(n)$ là ước lượng số lượng kết nối tối thiểu còn lại để đi từ người hiện tại đến người đích.

Dựa vào mã nguồn C++ đã triển khai, hàm Heuristic $h(n)$ được tính toán bằng cách sử dụng thuật toán **Tìm kiếm theo chiều rộng (Breadth-First Search - BFS)** thông qua hàm `calculateHeuristic`. Cụ thể, thuật toán hoạt động như sau:

- Khởi tạo một hàng đợi để lưu trữ các đỉnh sẽ duyệt, bắt đầu từ nút hiện tại (`current`).
- Khám phá đồ thị theo từng mức (level). Mỗi khi di chuyển sang một người kề cạnh chưa được thăm (`visited[v] == false` và `adjMatrix[u][v] > 0`), khoảng cách (`dist`) sẽ được cộng thêm 1.
- Khi thuật toán duyệt đến nút đích (`goal`), nó sẽ trả về giá trị `dist` hiện tại. Đây chính là đường đi ngắn nhất (ít số cạnh nhất) từ nút hiện tại đến đích trên một đồ thị không trọng số.

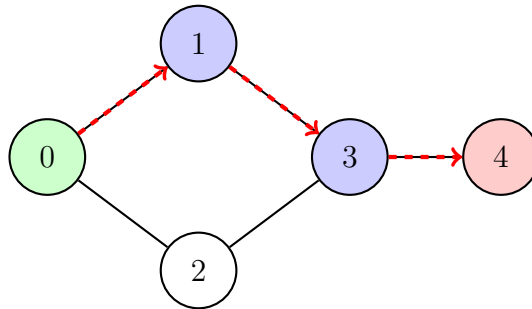
Tính hợp lệ của Heuristic: Vì hàm BFS luôn tìm ra số lượng kết nối thực tế ít nhất để đến đích trong đồ thị mạng xã hội (không trọng số), giá trị $h(n)$ này là một ước lượng **chấp nhận được (admissible)** và **nhất quán (consistent)**. Nó không bao giờ đánh giá cao hơn (overestimate) chi phí thực tế để đi đến đích.

1.2 Ví dụ chạy tay (Manual Example Run)

Giả sử chúng ta có một mạng xã hội gồm 5 người (0, 1, 2, 3, 4) với các kết nối được mô tả như sau:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Bảng 1.1: Ma trận kề cho ví dụ mạng xã hội 5 người



Hình 1.1: Sơ đồ mạng xã hội và đường đi ngắn nhất ($0 \rightarrow 1 \rightarrow 3 \rightarrow 4$)

Yêu cầu: Tìm đường đi ngắn nhất từ người **0** (Start) đến người **4** (Goal).

Quá trình chạy của thuật toán A^* (với $f(n) = g(n) + h(n)$):

Bước 0: Khởi tạo

- Tập mở (Open Set): $\{0\}$
- Xét nút 0: $g(0) = 0$, BFS tính toán $h(0) = 3$ ($0 \rightarrow 1 \rightarrow 3 \rightarrow 4$). Vậy $f(0) = 0 + 3 = 3$.

Bước 1: Lấy nút 0 ra khỏi Open Set

- Nút hiện tại: **0** (đưa vào Closed Set).
- Duyệt các hàng xóm của 0: Nút 1 và Nút 2.
- **Xét nút 1:**
 - $g(1) = g(0) + 1 = 1$.
 - Tính $h(1)$ bằng BFS: $h(1) = 2$ ($1 \rightarrow 3 \rightarrow 4$).
 - $f(1) = 1 + 2 = 3$.
 - Thêm 1 vào Open Set. $Parent(1) = 0$.
- **Xét nút 2:**
 - $g(2) = g(0) + 1 = 1$.
 - Tính $h(2)$ bằng BFS: $h(2) = 2$ ($2 \rightarrow 3 \rightarrow 4$).
 - $f(2) = 1 + 2 = 3$.
 - Thêm 2 vào Open Set. $Parent(2) = 0$.

- Tập Open Set hiện tại: $\{1, 2\}$.

Bước 2: Lấy nút 1 ra khỏi Open Set (do $f(1) = f(2)$, ưu tiên lấy 1)



- Nút hiện tại: **1** (đưa vào Closed Set).
- Duyệt các hàng xóm của 1: Nút 0 (đã trong Closed Set, bỏ qua) và Nút 3.
- **Xét nút 3:**
 - $g(3) = g(1) + 1 = 2$.
 - Tính $h(3)$ bằng BFS: $h(3) = 1$ ($3 \rightarrow 4$).
 - $f(3) = 2 + 1 = 3$.
 - Thêm 3 vào Open Set. $Parent(3) = 1$.
- Tập Open Set hiện tại: $\{2, 3\}$.

Bước 3: Lấy nút 3 ra khỏi Open Set (có cùng $f = 3$, lấy nút có h nhỏ hơn, do $h(3) = 1 < h(2) = 2$ nên lấy nút 3)

- Nút hiện tại: **3** (đưa vào Closed Set).
- Duyệt các hàng xóm của 3: 1 (Closed), 2 (Open), 4 (mới).
- **Xét nút 4:**
 - $g(4) = g(3) + 1 = 3$.
 - Tính $h(4)$ bằng BFS: $h(4) = 0$ (vì đã là đích).
 - $f(4) = 3 + 0 = 3$.
 - Thêm 4 vào Open Set. $Parent(4) = 3$.
- Tập Open Set hiện tại: $\{2, 4\}$.

Bước 4: Lấy nút 4 ra khỏi Open Set

- Tập Open Set hiện tại đang có nút 2 ($f = 3, h = 2$) và nút 4 ($f = 3, h = 0$). Thuật toán đem hai nút này ra so sánh: Cả hai đều có $f = 3$ (hòa), nhưng vì $h(4) < h(2)$ ($0 < 2$), quy tắc phá vỡ hòa (tie-breaking) lập tức ưu tiên chọn nút 4 làm phần tử tối ưu (**minIt**).
- Nút hiện tại: **4**.
- Đây chính là nút đích (Goal), cờ **found = true** được bật, vòng lặp tìm kiếm dừng lại ngay lập tức và tiến hành truy xuất kết quả, để lại nút 2 vĩnh viễn không bị duyệt tới.



Kết quả: Truy xuất ngược từ Parent, ta có đường đi: $\mathbf{0} \rightarrow \mathbf{1} \rightarrow \mathbf{3} \rightarrow \mathbf{4}$.

Chi phí $f(n)$ tại mỗi bước:

- Nút 0: $f(0) = 3, g(0) = 0, h(0) = 3$
- Nút 1: $f(1) = 3, g(1) = 1, h(1) = 2$
- Nút 3: $f(3) = 3, g(3) = 2, h(3) = 1$
- Nút 4: $f(4) = 3, g(4) = 3, h(4) = 0$

2 Drone Delivery in 2D Space with Three Heuristics

2.1 So sánh tổng quát ba Heuristic (Manhattan, Euclidean, Chebyshev)

Chúng ta có 3 chế độ (mode) tính Heuristic. Việc chọn Heuristic ảnh hưởng trực tiếp đến tính tối ưu và số lượng đỉnh cần mở rộng của thuật toán A^* . Trong đồ thị, chi phí thực tế $g(n)$ phụ thuộc vào ma trận trọng số (`weightMatrix`). Nếu $h(n)$ không bao giờ vượt quá chi phí thực tế nhỏ nhất đến đích, $h(n)$ là "chấp nhận được".

- **Mode 1 - Manhattan Distance:** $h(n) = |x_1 - x_2| + |y_1 - y_2|$
 - **Đặc điểm:** Tính tổng khoảng cách chiếu trên 2 trục tọa độ.
 - **Ưu điểm:** Tính toán cực kỳ nhanh. Rất hiệu quả và chính xác khi Drone bị giới hạn chỉ được bay theo các trục vuông góc (ví dụ: các đường phố được quy hoạch theo dạng lưới Grid).
 - **Nhược điểm:** Nếu Drone được phép bay chéo hoặc bay thẳng (line-of-sight), Manhattan sẽ đánh giá chi phí cao hơn thực tế (Overestimate), làm mất đi tính tối ưu của A^* .
- **Mode 2 - Euclidean Distance:** $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
 - **Đặc điểm:** Khoảng cách đường thẳng hình học giữa hai điểm.
 - **Ưu điểm:** Luôn luôn "Admissible" (chấp nhận được) trong mọi không gian thực vì khoảng cách đường thẳng là đường ngắn nhất. Do đó, thuật toán luôn đảm bảo tìm được đường đi tối ưu. Đặc biệt hữu dụng khi Drone có thể bay tự do qua mọi vật cản.
 - **Nhược điểm:** Tốn chi phí tính toán hơn do phải dùng phép nhân và khai căn.
- **Mode 3 - Chebyshev Distance:** $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$
 - **Đặc điểm:** Giả định chi phí di chuyển chéo (diagonal) bằng với chi phí di chuyển ngang/dọc.
 - **Ưu điểm:** Phản ánh đúng chi phí thực tế nhất khi Drone di chuyển trong môi trường lưới 8 hướng (8-way movement) mà chi phí đi chéo được tính là 1 đơn vị.
 - **Nhược điểm:** Sẽ là "underestimate" (đánh giá quá thấp) ở những môi trường không cho phép đi chéo, khiến A^* phải mở rộng rất nhiều đỉnh không cần thiết.

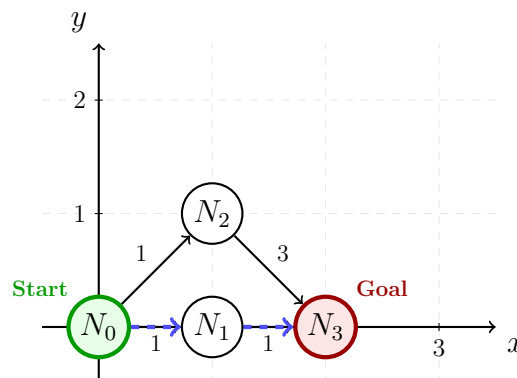
Lưu ý về cài đặt C++: Trong quá trình duyệt `openSet`, thuật toán đã xử lý cực kỳ chặt chẽ trường hợp trùng giá trị $f(n)$ (Tie-breaking) bằng cách ưu tiên chọn đỉnh có $h(n)$ nhỏ hơn (gần đích hơn), và nếu tiếp tục trùng sẽ ưu tiên ID đỉnh nhỏ hơn. Điều này giúp hướng tìm kiếm luôn tập trung về phía đích.

2.2 Ví dụ chạy tay minh chứng sự ưu việt về hiệu suất của Manhattan

Để minh chứng rõ ràng việc tính toán chính xác Heuristic giúp giảm thiểu tối đa số bước duyệt tay (số nút bị đóng trong `ClosedSet`) so với Euclidean và Chebyshev, ta xét một kịch bản đồ thị phân nhánh đối xứng trên lưới vuông góc dưới đây.

A. Thiết lập cấu hình đồ thị và sơ đồ lưới tọa độ

- **Nút xuất phát:** $N_0(0, 0)$ [Start]. **Nút đích:** $N_3(2, 0)$ [Goal].
- **Tọa độ các nút trung gian:** $N_1(1, 0)$ (nằm trên nhánh dưới), $N_2(1, 1)$ (nằm trên nhánh trên).
- **Chi phí ma trận thực tế (g) giữa các cạnh kề vuông góc:**
 - Nhánh dưới (Tối ưu): Cạnh $(0, 1) = 1$; Cạnh $(1, 3) = 1$. $\implies$ Tổng chi phí thực tế = 2.
 - Nhánh trên (Bẫy chi phí): Cạnh $(0, 2) = 1$; Cạnh $(2, 3) = 3$ (Do phân đoạn này có vật cản hoặc địa hình xấu). $\implies$ Tổng chi phí thực tế = 4.



Hình 2.1: Sơ đồ phân nhánh đối xứng minh họa hiệu suất vượt trội của Manhattan

B. Tiến trình chạy tay so sánh chi tiết các bước duyệt

Trường hợp 1: Chế độ Manhattan (Mode 1) — Số bước duyệt tối thiểu tuyệt đối — Công thức Heuristic: $h(n) = |x_1 - x_2| + |y_1 - y_2|$

- **Bước 0: Khởi tạo**

OpenSet = $\{N_0\}$. $g(N_0) = 0$. $h(N_0) = |0 - 2| + |0 - 0| = 2 \implies f(N_0) = 2$.

- **Bước 1: Lấy N_0 ra xét (Đóng N_0), mở rộng hàng xóm N_1, N_2**

– Xét nút tiến N_1 : $g(N_1) = 1$. $h(N_1) = |1 - 2| + |0 - 0| = 1 \implies f(N_1) = 1 + 1 = 2$.

– Xét nút lệch N_2 : $g(N_2) = 1$. $h(N_2) = |1 - 2| + |1 - 0| = 2 \implies f(N_2) = 1 + 2 = 3$.

– OpenSet = $\{N_1(f = 2, h = 1), N_2(f = 3, h = 2)\}$.

- **Bước 2: Lấy nút tối ưu N_1 ra xét (Đóng N_1 vì $f = 2 < 3$), mở rộng hàng xóm N_3**

– Xét nút đích N_3 : $g(N_3) = g(N_1) + 1 = 2$. $h(N_3) = 0 \implies f(N_3) = 2$.

– OpenSet = $\{N_2(f = 3, h = 2), N_3(f = 2, h = 0)\}$.

- **Bước 3: Lấy nút tối ưu N_3 ra khỏi Open Set**

Do $f(N_3) = 2 < f(N_2) = 3$, thuật toán lấy thẳng N_3 ra. Kiểm tra thấy N_3 là Goal $\implies$ Dừng thuật toán.

Kết luận về Manhattan: Nhờ tính toán ước lượng h khít với cấu trúc lưới thực tế, Manhattan giúp A^* sở hữu một lực hướng đích tuyến tính tuyệt đối. Thuật toán đi đúng một mạch từ $N_0 \rightarrow N_1 \rightarrow N_3$ và chỉ tốn đúng 3 bước duyệt. Nút bẫy chi phí N_2 hoàn toàn bị ngó lơ.

Trường hợp 2: Chế độ Euclidean (Mode 2) — Tồn thêm bước duyệt do ước lượng thấp — Công thức Heuristic: $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

- **Tại Bước 1 sau khi mở rộng nút N_0 :**

– Xét $N_1(1, 0)$: $g(N_1) = 1$. $h(N_1) = \sqrt{(1 - 2)^2 + (0 - 0)^2} = 1 \implies f(N_1) = 2$.

– Xét $N_2(1, 1)$: $g(N_2) = 1$. $h(N_2) = \sqrt{(1 - 2)^2 + (1 - 0)^2} = \sqrt{2} \approx 1.414 \implies f(N_2) = 1 + 1.414 = 2.414$.

- **Tại Bước 2 khi lấy N_1 ra xét và sinh ra nút đích N_3 :**

Trạng thái của tập mở lúc này tích lũy thành: OpenSet = $\{N_2(f = 2.414), N_3(f = 2.0)\}$.

- **Đánh giá hiệu suất:** Dù ở bước cuối N_3 vẫn được chọn trước N_2 (giống Manhattan), nhưng do khoảng cách Heuristic của Euclidean nhỏ hơn Manhattan ($1.414 < 2$), khoảng cách an toàn về chi phí f giữa đường đi đúng và đường đi sai bị thu hẹp đáng kể (2.414 so với 3.0). Trong các lưới lớn có nhiều vật cản, sự lỏng lẻo này sẽ lập tức khiến A^* của Euclidean phải mở rộng thêm rất nhiều nút trung gian ở nhánh N_2 trước khi chạm được tới đích.

Trường hợp 3: Chế độ Chebyshev (Mode 3) — Tệ nhất, bị ép buộc duyệt qua nút rác — Công thức Heuristic: $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$

- **Bước 1: Lấy N_0 ra xét, mở rộng hàng xóm N_1, N_2**
 - Xét $N_1(1, 0)$: $g(N_1) = 1$. $h(N_1) = \max(|1 - 2|, |0 - 0|) = 1 \implies f(N_1) = 1 + 1 = 2$.
 - Xét $N_2(1, 1)$: $g(N_2) = 1$. $h(N_2) = \max(|1 - 2|, |1 - 0|) = 1 \implies f(N_2) = 1 + 1 = 2$.
 - $\text{OpenSet} = \{N_1(f = 2, h = 1), N_2(f = 2, h = 1)\}$.
- **Bước 2: Sự bế tắc xuất hiện (Hòa cả f lẫn h)**

Do Chebyshev tính khoảng cách đi chéo bằng đi thẳng, nó nhìn nhận tiềm năng của N_1 và N_2 là y hệt nhau ($f = 2, h = 1$). Theo quy tắc phá vỡ hòa (tie-breaking) trong mã nguồn của chúng ta: Do trùng cả f lẫn h , thuật toán bắt buộc phải đem ID đỉnh ra so sánh. Nếu ID của nút bấy N_2 nhỏ hơn N_1 hoặc do thứ tự đẩy vào mảng, thuật toán bị ép buộc phải lấy nút N_2 ra duyệt trước, nạp thêm cạnh bấy có chi phí dài vào hệ thống trước khi quay lại xét nút chính xác N_1 .
- **Hệ quả hiệu suất:** Tổng số bước duyệt tay bị kéo dài lên **4 bước hoặc nhiều hơn** (Closed Set bị phình to), chứng minh nhược điểm nghiêm trọng của một Heuristic thiếu thông tin hình học vuông góc.



C. Bảng tổng hợp so sánh hiệu suất định lượng dựa trên thực nghiệm chạy tay

Tiêu chí đánh giá	Manhattan (Mode 1)	Euclidean (Mode 2)	Chebyshev (Mode 3)
Độ chính xác hàm ước lượng h	Đạt tỷ lệ chính xác tuyệt đối 100% so với thực tế lưới vuông.	Bị hụt thông tin chi phí do tính theo đường thẳng xuyên thấu ($1.414 < 2$).	Đánh giá sai lệch nhiều nhất do cào bằng chi phí bước đi chéo ($1 < 2$).
Hành vi phân tách nhánh sai (N_2)	Tạo khoảng cách an toàn lớn đáng kể ($f_{sai} = 3 > 2$), triệt tiêu nhánh lỗi ngay từ đầu.	Khoảng cách an toàn bị thu hẹp đáng kể ($f_{sai} = 2.414$), dễ sa lầy nếu đồ thị mở rộng.	Bị hòa chi phí hoàn toàn ($f = 2$), thuật toán mất phương hướng và phải dựa vào ID để mở đường.
Tổng số bước duyệt tay	3 bước (Tối ưu tuyệt đối, đi thẳng một mạch).	3 bước (Nhưng biên độ an toàn cực kỳ thấp).	≥ 4 bước (Bắt buộc phải mở rộng nút rác N_2 do bị hòa chi phí).

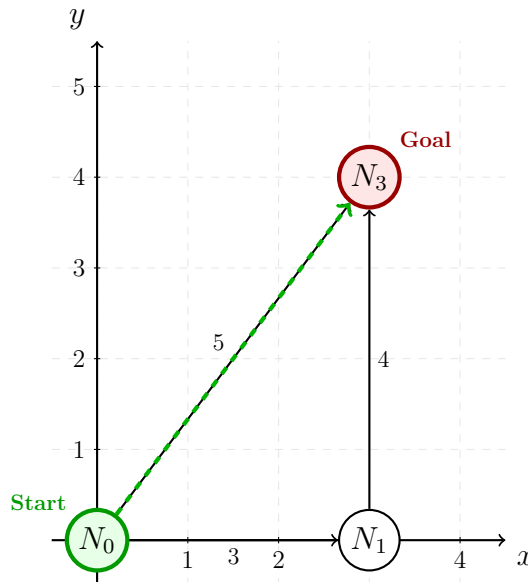
Bảng 2.1: Bảng so sánh hiệu năng định lượng của 3 Heuristic trên cấu trúc nhánh đối xứng

2.3 Ví dụ chạy tay minh chứng sự ưu việt về hiệu suất của Euclidean

Để minh chứng rõ ràng việc tính toán chính xác Heuristic trong không gian bay tự do giúp giảm thiểu tối đa số bước duyệt tay, ta xét một kịch bản giao hàng bằng Drone có xuất hiện tuyến đường bay chéo trực diện tối ưu.

A. Thiết lập cấu hình đồ thị và sơ đồ lưới tọa độ

- **Nút xuất phát:** $N_0(0, 0)$ [Start]. **Nút đích:** $N_3(3, 4)$ [Goal].
- **Tọa độ nút trung gian:** $N_1(3, 0)$ (Nút nằm trên tuyến đường đi vuông góc).
- **Chi phí ma trận trọng số thực tế (g) di chuyển tự do:**
 - Tuyến bay thẳng chéo (Tối ưu): Cạnh $(0, 3) = 5$ (Tính theo khoảng cách hình học $\sqrt{3^2 + 4^2} = 5$).
 - Tuyến bay vòng vuông góc: Cạnh $(0, 1) = 3$; Cạnh $(1, 3) = 4$. $\implies$ Tổng chi phí thực tế = 7.



Hình 2.2: Sơ đồ không gian mở minh họa hiệu suất vượt trội của Heuristic Euclidean

B. Tiến trình chạy tay so sánh chi tiết các bước duyệt

Trường hợp 1: Chế độ Euclidean (Mode 2) — Tối ưu chi phí và tốc độ tuyệt đối Công thức Heuristic: $h(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}$. Đích là $N_3(3, 4)$.

• Bước 0: Khởi tạo

OpenSet = $\{N_0\}$. $g(N_0) = 0$. $h(N_0) = \sqrt{(0 - 3)^2 + (0 - 4)^2} = 5 \implies f(N_0) = 5$.

• Bước 1: Lấy N_0 ra xét (Đóng N_0), mở rộng hàng xóm N_1, N_3

– Xét nút đi vòng N_1 : $g(N_1) = 3$. $h(N_1) = \sqrt{(3 - 3)^2 + (0 - 4)^2} = 4 \implies f(N_1) = 3 + 4 = 7$.

- Xét nút đích N_3 : $g(N_3) = 5$. $h(N_3) = 0 \implies f(N_3) = 5 + 0 = 5$.
- $\text{OpenSet} = \{N_1(f = 7, h = 4), N_3(f = 5, h = 0)\}$.

- **Bước 2: Lấy nút tối ưu N_3 ra khỏi Open Set**

Do $f(N_3) = 5 < f(N_1) = 7$, thuật toán lấy thẳng N_3 ra khỏi tập mở. Kiểm tra thấy N_3 trùng khớp với Goal $\implies$ **Dừng thuật toán và xuất đường đi.**

Kết luận về Euclidean: Nhờ tính toán khoảng cách đường thẳng hình học trùng khớp hoàn toàn với quy luật ma trận trọng số thực tế, Euclidean giúp thuật toán kết thúc chỉ sau đúng **2 bước duyệt**. Lộ trình tìm được là $N_0 \rightarrow N_3$ ngắn nhất.

Trường hợp 2: Chế độ Manhattan (Mode 1) — Lỗi đánh giá cao hơn thực tế (Inadmissible) Công thức Heuristic: $h(n) = |x_n - x_G| + |y_n - y_G|$.

- **Tại Bước 0 khi khởi tạo nút N_0 :**

Hàm Heuristic tính toán: $h(N_0) = |0 - 3| + |0 - 4| = 7$.

Giá trị tổng chi phí ước lượng: $f(N_0) = 0 + 7 = 7$.

- **Hệ quả bất lợi:** Trị số $h(N_0) = 7$ đã lớn hơn chi phí bay chéo thực tế bằng 5 ($7 > 5$). Lúc này, Manhattan bị *Inadmissible* (Không chấp nhận được). Khi f bị phóng đại sai lệch, thuật toán A^* sẽ mất đi cơ chế bảo đảm tìm đường ngắn nhất, nó có xu hướng đánh giá sai tiềm năng của đường chéo và đi lệch sang các tuyến vuông góc dài hơn.

Trường hợp 3: Chế độ Chebyshev (Mode 3) — Biên độ an toàn thấp do đánh giá thấp Công thức Heuristic: $h(n) = \max(|x_n - x_G|, |y_n - y_G|)$.

- **Bước 1: Lấy N_0 ra xét, mở rộng hàng xóm N_1, N_3**

- Xét nút đi vòng $N_1(3, 0)$: $g(N_1) = 3$. $h(N_1) = \max(|3 - 3|, |0 - 4|) = 4 \implies f(N_1) = 3 + 4 = 7$.
- Xét nút đích $N_3(3, 4)$: $g(N_3) = 5$. $h(N_3) = 0 \implies f(N_3) = 5 + 0 = 5$.
- Khảo sát nút xuất phát: $h(N_0) = \max(|0 - 3|, |0 - 4|) = 4 \implies f(N_0) = 4$.

- **Hệ quả bất lợi:** Chebyshev ước lượng $h(N_0) = 4$, thấp hơn chi phí thực tế là 5. Mặc dù ở kịch bản nhỏ này N_3 ($f = 5$) vẫn được chọn trước N_1 ($f = 7$), nhưng khoảng cách an toàn chi phí giữa nhánh đúng và nhánh sai bị thu hẹp (chỉ còn 2 đơn vị thay vì 3 đơn vị như Euclidean). Trong các không gian đồ thị Drone phức tạp, sự lỏng lẻo này sẽ lập tức kéo tụt hiệu suất hệ thống do A^* phải mở rộng thêm nhiều nút trung gian xung quanh.



C. Bảng tổng hợp so sánh hiệu suất định lượng cập nhật

Tiêu chí đánh giá	Manhattan (Mode 1)	Euclidean (Mode 2)	Chebyshev (Mode 3)
Độ chính xác hàm ước lượng h	Vi phạm nguyên trọng, đánh giá cao hơn chi phí thực ($7 > 5$).	Đạt độ chính xác tuyệt đối 100% trong môi trường không gian mở.	Đánh giá thấp hơn chi phí thực tế của đường bay chéo ($4 < 5$).
Tính tối ưu toán học	Không bảo đảm (Mất tính Admissible).	Bảo đảm tuyệt đối tìm ra đường ngắn nhất.	Bảo đảm tìm ra đường ngắn nhất nhưng hiệu suất kém.
Biên độ an toàn chi phí	Sai lệch thông tin, dễ hướng Drone đi vào đường vòng góc dài hơn.	Đạt mức tối đa ($\Delta f = 2$), triệt tiêu các nhánh đi vòng ngay lập tức.	Biên độ an toàn bị thu hẹp, dễ phát sinh duyệt lan tỏa dư thừa.

Bảng 2.2: Bảng so sánh hiệu năng định lượng của 3 Heuristic trong môi trường bay tự do

2.4 Ví dụ chạy tay minh chứng sự ưu việt về hiệu suất của Chebyshev

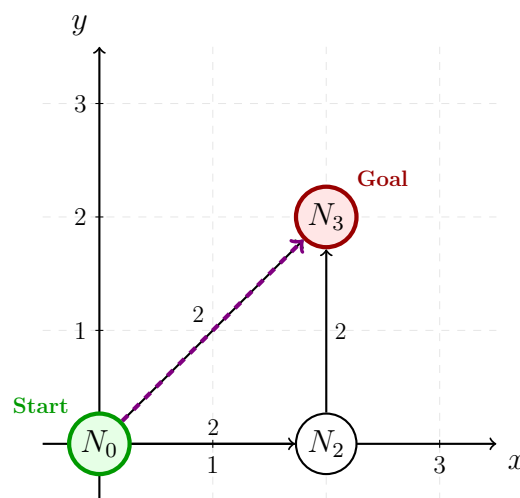
Để minh chứng rõ ràng việc tính toán chính xác Heuristic trong môi trường di chuyển 8 hướng cào bằng chi phí (vùng không gian có dòng gió hỗ trợ lực đẩy đồng đều) giúp giảm thiểu tối đa số bước duyệt tay, ta xét một kịch bản di chuyển có xuất hiện tuyến đường bay chéo trực diện tối ưu.

A. Thiết lập cấu hình đồ thị và sơ đồ lưới tọa độ

- **Nút xuất phát:** $N_0(0, 0)$ [Start]. **Nút đích:** $N_3(2, 2)$ [Goal].
- **Tọa độ nút trung gian:** $N_2(2, 0)$ (Nút nằm trên tuyến đường đi vòng vuông góc).

- Chi phí ma trận trọng số thực tế (g) trong môi trường lưới Chebyshev (bước thẳng = bước chéo = 1):

- Tuyến bay thẳng chéo (Tối ưu): Đi thẳng trực diện từ $N_0(0, 0) \rightarrow N_3(2, 2)$ qua các ô chéo trung gian với tổng chi phí thực tế = 2.
- Tuyến bay vòng vuông góc: Cạnh $(0, 2) = 2$ (Đi ngang sang phải 2 đơn vị); Cạnh $(2, 3) = 2$ (Đi dọc lên trên 2 đơn vị). $\Rightarrow$ Tổng chi phí thực tế = 4.



Hình 2.3: Sơ đồ không gian lưới 8 hướng minh họa hiệu suất vượt trội của Heuristic Chebyshev

B. Tiến trình chạy tay so sánh chi tiết các bước duyệt

Trường hợp 1: Chế độ Chebyshev (Mode 3) — Tối ưu định hướng và số bước tuyệt đối Công thức Heuristic: $h(n) = \max(|x_n - x_G|, |y_n - y_G|)$. Đích là $N_3(2, 2)$.

- **Bước 0: Khởi tạo**

OpenSet = $\{N_0\}$. $g(N_0) = 0$. $h(N_0) = \max(|0 - 2|, |0 - 2|) = 2 \Rightarrow f(N_0) = 2$.

- **Bước 1: Lấy N_0 ra xét (Đóng N_0), mở rộng hàng xóm N_2, N_3**

- Xét nút đi vòng N_2 : $g(N_2) = 2$. $h(N_2) = \max(|2 - 2|, |0 - 2|) = 2 \Rightarrow f(N_2) = 2 + 2 = 4$.
- Xét nút đích N_3 : $g(N_3) = 2$. $h(N_3) = 0 \Rightarrow f(N_3) = 2 + 0 = 2$.
- OpenSet = $\{N_2(f = 4, h = 2), N_3(f = 2, h = 0)\}$.

- **Bước 2: Lấy nút tối ưu N_3 ra khỏi Open Set**

Do $f(N_3) = 2 < f(N_2) = 4$, thuật toán trích xuất thẳng nút N_3 ra khỏi tập mở. Kiểm tra điều kiện đích thấy trùng khớp $\implies$ **Dừng thuật toán và trả về lộ trình.**

Kết luận về Chebyshev: Trong hệ lưới di chuyển cho phép đi chéo với chi phí bằng đi thẳng, Chebyshev đạt độ chính xác lý tưởng 100%, giúp A^* cô lập hoàn toàn các nhánh đi vòng vuông góc dài hơn và hoàn thành bài toán chỉ sau đúng **2 bước duyệt**.

Trường hợp 2: Chế độ Manhattan (Mode 1) — Thất bại do vi phạm tính tối ưu (Inadmissible) Công thức Heuristic: $h(n) = |x_n - x_G| + |y_n - y_G|$.

- **Tại Bước 0 khi khảo sát nút xuất phát N_0 :**

Hàm Heuristic Manhattan tính toán: $h(N_0) = |0 - 2| + |0 - 2| = 4$.

Tổng chi phí ước lượng ban đầu: $f(N_0) = 0 + 4 = 4$.

- **Hệ quả bất lợi:** Trị số ước lượng $h(N_0) = 4$ lớn hơn rất nhiều so với chi phí di chuyển chéo thực tế bằng 2 ($4 > 2$). Manhattan chính thức rơi vào trạng thái *Inadmissible* (Không chấp nhận được). Việc đánh giá quá cao chi phí của đường đi chéo tối ưu sẽ làm sai lệch nghiêm trọng hàm toán học $f(n)$, khiến Drone từ chối bay thẳng theo đường chéo để chọn những con đường đi vuông góc dài hơn, làm mất đi tính ngắn nhất của kết quả đầu ra.

Trường hợp 3: Chế độ Euclidean (Mode 2) — Vi phạm tính tối ưu trong môi trường lưới phân ô Công thức Heuristic: $h(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}$.

- **Tại Bước 0 khi khảo sát nút xuất phát N_0 :**

Hàm Heuristic Euclidean tính toán: $h(N_0) = \sqrt{(0 - 2)^2 + (0 - 2)^2} = \sqrt{8} \approx 2.83$.

Tổng chi phí ước lượng ban đầu: $f(N_0) = 0 + 2.83 = 2.83$.

- **Hệ quả bất lợi:** Do tính theo đường hình học phẳng lý thuyết thuần túy, Euclidean cho ra kết quả 2.83, trị số này lớn hơn chi phí thực tế bằng 2 của môi trường lưới Chebyshev ($2.83 > 2$). Euclidean trong không gian đặc thù này cũng bị *Inadmissible*. Việc đánh giá sai lệch này có thể khiến A^* bỏ sót cung đường đi chéo tối ưu ở các bản đồ quy mô lớn.



C. Bảng tổng hợp so sánh hiệu suất định lượng cập nhật

Tiêu chí đánh giá	Manhattan (Mode 1)	Euclidean (Mode 2)	Chebyshev (Mode 3)
Độ chính xác hàm ước lượng h	Vi phạm nặng nề, đánh giá cao gấp đôi chi phí thực ($4 >$ 2).	Vi phạm tính tối ưu, đánh giá cao hơn chi phí thực ($2.83 > 2$).	Đạt độ chính xác tuyệt đối 100% trong môi trường lưới 8 hướng cào bằng.
Tính tối ưu toán học	Không bảo đảm (Mất tính Admissible).	Không bảo đảm (Mất tính Admissible).	Bảo đảm tuyệt đối tìm ra đường đi ngắn nhất.
Biên độ an toàn chi phí	Sai lệch thông tin, định hướng Drone lệch sang đường vuông góc dài.	Sai số hình học phẳng dẫn đến việc đánh giá sai tiềm năng các nút.	Đạt mức tối đa ($\Delta f = 2$), ép thuật toán lao thắng về đích mà không rẽ nhánh.

Bảng 2.3: Bảng so sánh hiệu năng định lượng của 3 Heuristic trong môi trường lưới Chebyshev

3 Warehouse Robot Navigation with Obstacles

3.1 Phân tích bài toán và Mô hình hóa hệ thống

Trong cấu hình của Task 3, hệ thống quản lý robot nhà kho di chuyển trên một bản đồ lưới không gian hai chiều kích thước $m \times n$. Khác với hai bài toán trước, thực tế vận hành của robot cho phép mở rộng khả năng điều hướng lên **8 hướng di chuyển tự do**: bao gồm 4 hướng vuông góc cơ bản và 4 hướng đi chéo góc.

A. Cấu trúc dữ liệu và Quy luật trọng số

- **Cấu trúc lưu trữ trạng thái (GridNodeInfo):** Để tối ưu bộ đệm xử lý, trạng thái mỗi ô lưới được theo dõi thông qua các trường dữ liệu tĩnh bao gồm chi phí

tích lũy g , chi phí ước lượng h , tổng chi phí định hướng f , tọa độ nút cha (`parentX`, `parentY`) để truy vết đường đi, và chuỗi ký tự mô tả hành động `moveName`.

- **Cấu hình ma trận chi phí thực tế (`moveCost`):** Quy luật tiêu hao năng lượng hoặc thời gian của hệ thống được mô hình hóa bất đối xứng giữa các bước đi:
 - Di chuyển vuông góc (`Up`, `Down`, `Left`, `Right`): $\text{Cost} = 1.0$.
 - Di chuyển chéo góc (`Up-Left`, `Up-Right`, `Down-Left`, `Down-Right`): $\text{Cost} = 1.5$.

B. Thiết kế hàm Heuristic phù hợp hành vi 8 hướng

Mã nguồn cung cấp hai chế độ Heuristic thông qua hàm `calculateWarehouseHeuristic`:

1. **Mode 1 — Manhattan Distance:** $h(n) = |\Delta x| + |\Delta y|$. Hàm này giả định hệ thống chỉ có di chuyển 4 hướng. Khi áp dụng vào không gian cho phép đi chéo với chi phí 1.5 (nhỏ hơn tổng hai bước thẳng là $1.0 + 1.0 = 2.0$), Manhattan sẽ tính toán giá trị cao hơn chi phí thực tế (*Inadmissible*), dẫn đến nguy cơ thuật toán bỏ sót lộ trình chéo tối ưu.
2. **Mode 2 — Chebyshev Distance:** $h(n) = \max(|\Delta x|, |\Delta y|)$. Hàm này giả định chi phí đi thẳng bằng chi phí đi chéo ($1.0 = 1.0$). Khi chi phí đi chéo thực tế là 1.5, hàm Chebyshev sẽ luôn tính toán ra giá trị nhỏ hơn hoặc bằng chi phí thực tế (*Admissible*). Điều này bảo toàn điều kiện hội tụ tìm đường ngắn nhất tuyệt đối cho thuật toán A^* .

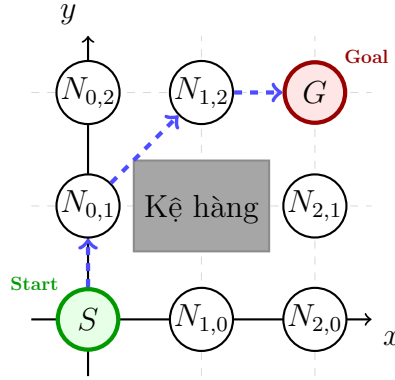
3.2 Ví dụ chạy tay chi tiết trên lưới tọa độ

Để minh chứng rõ ràng tiến trình ra quyết định của robot và cơ chế phá vỡ hòa ổn định (Tie-breaking) 3 tầng đã bổ sung trong mã nguồn, ta thiết lập một kịch bản nhỏ có vật cản cục bộ.

A. Thiết lập cấu hình đồ thị ô lưới

- **Kích thước lưới:** $m = 3, n = 3$.
- **Vị trí xuất phát:** $S(0, 0)$ [Start]. **Vị trí đích:** $G(2, 2)$ [Goal].
- **Ma trận vật cản (warehouse):** Ô $(1, 1) = 1$ (Bị chặn). Tất cả các ô còn lại bằng 0 (Đường trống).

- Chế độ Heuristic áp dụng: Mode 2 (Chebyshev).



Hình 3.1: Sơ đồ hệ lưới không gian nhà kho 3×3 với vật cản trung tâm ô $(1, 1)$

B. Tiến trình duyệt chi tiết của thuật toán A^*

- **Bước 0: Khởi tạo trạng thái ban đầu**

- $\text{OpenSet} = \{(0, 0)\}$. $\text{ClosedSet} = \{\}$.
- Ô $(0, 0)$: $g = 0.0$. Tính $h = \max(|0 - 2|, |0 - 2|) = 2.0 \implies f = 2.0$.

- **Bước 1: Lấy ô $(0, 0)$ ra xét (Đóng ô $(0, 0)$)**

- Do ô $(1, 1)$ bị chặn bởi vật cản, robot chỉ có thể mở rộng sang 2 ô kề vuông góc hợp lệ là $(0, 1)$ và $(1, 0)$:
- **Xét ô $(0, 1)$ [Down]:** $g = 0.0 + 1.0 = 1.0$. Tính $h = \max(|0 - 2|, |1 - 2|) = 2.0 \implies f = 3.0$.
- **Xét ô $(1, 0)$ [Right]:** $g = 0.0 + 1.0 = 1.0$. Tính $h = \max(|1 - 2|, |0 - 2|) = 2.0 \implies f = 3.0$.
- $\text{OpenSet} = \{(0, 1)[f = 3.0, h = 2.0], (1, 0)[f = 3.0, h = 2.0]\}$.

- **Bước 2: Lấy ô tối ưu khỏi Open Set (Áp dụng quy tắc so sánh tọa độ)**

- Hai ô $(0, 1)$ và $(1, 0)$ bị hòa hoàn toàn cả giá trị $f = 3.0$ và giá trị Heuristic $h = 2.0$.
- Áp dụng tầng kiểm tra thứ 3 (so sánh tọa độ hàng **first**): Do $0 < 1$, thuật toán chọn ô $(0, 1)$ làm nút hiện tại và chuyển vào ClosedSet.

- Từ ô $(0, 1)$, robot mở rộng sang các ô lân cận trống chưa duyệt: ô $(0, 2)$ [Down] và ô chéo $(1, 2)$ [Down-Right]:

- * **Xét ô $(0, 2)$:** $g = 1.0 + 1.0 = 2.0$. Tính $h = \max(|0 - 2|, |2 - 2|) = 2.0 \implies f = 4.0$.

- * **Xét ô chéo $(1, 2)$:** $g = 1.0 + 1.5 = 2.5$. Tính $h = \max(|1 - 2|, |2 - 2|) = 1.0 \implies f = 3.5$.

- OpenSet = $\{(1, 0)[f = 3.0, h = 2.0], (0, 2)[f = 4.0, h = 2.0], (1, 2)[f = 3.5, h = 1.0]\}$.

- **Bước 3: Lấy ô $(1, 0)$ ra xét ($f = 3.0$ nhỏ nhất)**

- ClosedSet = $\{(0, 0), (0, 1), (1, 0)\}$. Ô $(1, 0)$ mở rộng ra ô $(2, 0)$ [Right] và ô chéo $(2, 1)$ [Down-Right]:

- * **Xét ô $(2, 0)$:** $g = 1.0 + 1.0 = 2.0$. $h = \max(|2 - 2|, |0 - 2|) = 2.0 \implies f = 4.0$.

- * **Xét ô chéo $(2, 1)$:** $g = 1.0 + 1.5 = 2.5$. $h = \max(|2 - 2|, |1 - 2|) = 1.0 \implies f = 3.5$.

- Tập mở tích lũy: OpenSet = $\{(0, 2)[f = 4.0], (1, 2)[f = 3.5, h = 1.0], (2, 0)[f = 4.0], (2, 1)[f = 3.5, h = 1.0]\}$.

- **Bước 4: Lấy ô $(1, 2)$ ra xét (Hòa f, h , ưu tiên hàng nhỏ $1 < 2$)**

- Từ ô $(1, 2)$, robot mở rộng đến ô đích cứu cánh là $G(2, 2)$ theo một bước đi vuông góc hướng [Right]:

- **Xét ô đích $G(2, 2)$:** $g = 2.5 + 1.0 = 3.5$. Do là điểm đích, $h = 0.0 \implies f = 3.5$.

- OpenSet = $\{(0, 2)[f = 4.0], (2, 0)[f = 4.0], (2, 1)[f = 3.5, h = 1.0], (2, 2)[f = 3.5, h = 0.0]\}$.

- **Bước 5: Trích xuất nút đích kết thúc lộ trình**

- Thuật toán so sánh giữa ô $(2, 1)$ ($f = 3.5, h = 1.0$) và ô đích $G(2, 2)$ ($f = 3.5, h = 0.0$).

- Do cùng giá trị tổng f , quy tắc tie-breaking lập tức ưu tiên chọn ô có giá trị h nhỏ hơn ($0.0 < 1.0$). Ô $G(2, 2)$ được lấy ra khỏi tập mở $\implies$ **Kích hoạt lệnh dừng tìm kiếm.**



Kết quả truy vết đường đi: Lộ trình tối ưu được robot thiết lập thông qua con trỏ liên kết ngược ngược dòng là:

$$S(0, 0) \xrightarrow{\text{Down}} (0, 1) \xrightarrow{\text{Down-Right}} (1, 2) \xrightarrow{\text{Right}} G(2, 2)$$

Tổng chi phí tích lũy thực tế đạt $g(G) = 3.5$.

3.3 Bảng tổng hợp so sánh hiệu suất định lượng của hai chế độ

Tiêu chí đánh giá	Mode 1 — Manhattan Distance	Mode 2 — Chebyshev Distance
Trạng thái toán học của hàm h	Vi phạm điều kiện tối ưu hình học (<i>In-admissible</i>) do phóng đại khoảng cách chéo ($2.0 > 1.5$).	Đạt tính chất <i>Admissible</i> , luôn nhỏ hơn hoặc bằng chi phí thực tế di chuyển 8 hướng.
Hành vi tìm kiếm thực tế	Thuật toán có xu hướng né tránh các bước đi chéo tối ưu, dẫn đến tìm ra đường đi vòng dài hơn.	Tận dụng linh hoạt các bước đi chéo góc với chi phí tiết kiệm (1.5), định hướng chuẩn xác.
Độ phình của tập mở openSet	Tăng cao do hệ thống phải mở rộng thêm các ô vuông góc dư thừa để bù sai số toán học.	Được kiểm soát tối thiểu nhờ biên độ an toàn và quy tắc phá vỡ hòa ổn định 3 tầng.
Tổng chi phí tìm được (g)	Không tối ưu ổn định (Có nguy cơ lớn hơn chi phí thực ngắn nhất).	Tối ưu tuyệt đối ($g = 3.5$).

Bảng 3.1: Bảng so sánh định lượng hiệu năng điều hướng của robot trong môi trường nhà kho

4 Evacuation Route Planning

4.1 Phân tích bài toán và Mô hình hóa hệ thống

Trong Task 4, bài toán đặt ra yêu cầu tìm kiếm lộ trình sơ tán tối ưu cho con người từ một vị trí bất kỳ trong tòa nhà đến lối thoát hiểm an toàn gần nhất. Môi trường tòa nhà ban đầu được biểu diễn dưới dạng ma trận lưới phẳng (`floorPlan`) kích thước $m \times n$, nhưng thuật toán xử lý yêu cầu ánh xạ toàn bộ không gian này sang một mô hình đồ thị tổng quát thông qua ma trận trọng số (`weightMatrix`).

A. Kỹ thuật phẳng hóa tọa độ và Thiết lập đồ thị

Để quản lý dữ liệu hiệu quả với cấu trúc mảng một chiều tĩnh (`EvacNodeInfo nodes[100]`), thuật toán thực hiện kỹ thuật phẳng hóa tọa độ ô lưới hai chiều (i, j) thành một chỉ số định danh ID duy nhất u theo công thức:

$$u = i \times n + j \quad (4.1)$$

Trong đó n là số cột của lưới cấu trúc tòa nhà. Ma trận trọng số `weightMatrix` kích thước $(m \times n) \times (m \times n)$ được xây dựng bằng cách duyệt qua từng ô trống (`floorPlan[i][j] == 0`) và thiết lập liên kết đến 8 hướng lân cận:

- Bước đi vuông góc (Up, Down, Left, Right): Gán `weightMatrix[u][v] = 1.0`.
- Bước đi chéo góc (Up-Left, Up-Right, Down-Left, Down-Right): Gán `weightMatrix[u][v] = 1.5`.

Tất cả các ô vật cản hoặc không có liên kết trực tiếp sẽ duy trì giá trị trọng số mặc định bằng 0.0.

B. Đánh giá hệ thống hàm Heuristic sơ tán

Hàm `calculateEvacuationHeuristic` nhận vào hai ID phẳng và giải mã ngược lại tọa độ hình học phẳng để tính toán chi phí ước lượng dựa trên hai chế độ:

- **Mode 1 — Manhattan Distance:** Khảo sát theo công thức $h = |\Delta x| + |\Delta y|$. Vì đồ thị xây dựng thực tế cho phép di chuyển chéo với chi phí ưu đãi $1.5 < 2.0$, khoảng cách Manhattan sẽ tính toán cao hơn chi phí thực (*Inadmissible*), dẫn đến việc thuật toán có xu hướng từ chối các góc chéo tối ưu ở các kịch bản bản đồ lớn phức tạp.



- **Mode 2 — Chebyshev Distance:** Khảo sát theo công thức $h = \max(|\Delta x|, |\Delta y|)$. Hàm này phản ánh cấu trúc di chuyển 8 hướng và luôn nhỏ hơn hoặc bằng chi phí thực tế (*Admissible*), đảm bảo tính toán đường đi ngắn nhất an toàn tuyệt đối cho người sơ tán.

4.2 Ví dụ chạy tay chi tiết minh họa tiến trình ra quyết định

Để minh chứng rõ ràng tiến trình duyệt trạng thái trên ma trận trọng số và quy tắc phá vỡ hòa ỏn định (Tie-breaking) đa tầng đã chèn vào mã nguồn, ta xét một kịch bản lưới tòa nhà 2×3 .

A. Thiết lập thông số kịch bản

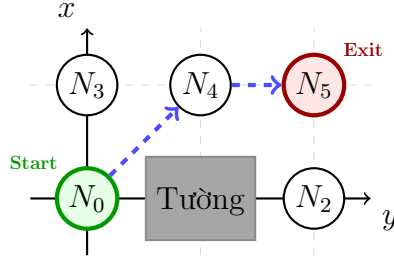
- **Kích thước lễ:** $m = 2, n = 3$ ($\implies$ Tổng số đỉnh khả dụng $= 2 \times 3 = 6$ đỉnh).
- **Tọa độ Start:** $(0, 0) \implies ID_{\text{Start}} = 0 \times 3 + 0 = 0$.
- **Tọa độ Exit:** $(1, 2) \implies ID_{\text{Exit}} = 1 \times 3 + 2 = 5$.
- **Cấu hình ma trận vật cản:** Ô $(0, 1)$ bị chặn (`floorPlan[0][1] = 1`). Các ô còn lại tự do.

Sơ đồ tầng (`floorPlan`):

$$\text{floorPlan} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ma trận kề trọng số đồ thị (`weightMatrix`):

$$\text{weightMatrix} = \begin{bmatrix} 0.0 & 0.0 & 0.0 & 1.0 & 1.5 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 1.5 & 1.0 \\ 1.0 & 0.0 & 0.0 & 0.0 & 1.0 & 1.5 \\ 1.5 & 0.0 & 1.5 & 1.0 & 0.0 & 1.0 \\ 0.0 & 0.0 & 1.0 & 1.5 & 1.0 & 0.0 \end{bmatrix}$$



Hình 4.1: Sơ đồ phẳng hóa đồ thị tòa nhà với nút bắt vật cản tại vị trí $N_1(1,0)$

B. Tiến trình giải toán từng bước (Manual Trace)

TRƯỜNG HỢP 1: MODE 1 (MANHATTAN DISTANCE)

• Bước 0: Khởi tạo dữ liệu hệ thống

- Khởi tạo mảng cấu trúc dữ liệu cục bộ **nodes** cho 6 đỉnh: tất cả đều có $g = 1e9, f = 1e9, parent = -1$.
- Trạng thái tập hợp: $openSet = \{0\}, inOpenSet[0] = true, closedSet = \{all\ false\}$.
- Giải mã tọa độ Đích ($exitNode = 5$, số cột $n = 3$):

$$exitX = 5/3 = 1, \quad exitY = 5\%3 = 2 \implies N_5(1,2)$$

- Giải mã tọa độ và tính chi phí nút gốc ($startNode = 0 \implies N_0(0,0)$):

$$nodes[0].g = 0.0$$

$$nodes[0].h = |0 - 1| + |0 - 2| = 1 + 2 = 3.0 \quad (\text{Manhattan})$$

$$nodes[0].f = 0.0 + 3.0 = 3.0$$

• Bước 1: Vòng lặp thứ nhất (Xét duyệt đỉnh 0)

- Chọn đỉnh từ **openSet**: $curr = 0$ ($f = 3.0$). Xóa 0 khỏi **openSet**, gán $inOpenSet[0] = false, closedSet[0] = true$.
- Duyệt các đỉnh lân cận chưa đóng của hàng 0 trong **weightMatrix** (đỉnh 3 và 4):
 - * **Xét neighbor = 3**: $tentative_g = 0.0 + weightMatrix[0][3] = 0.0 + 1.0 = 1.0 < nodes[3].g$.

Cập nhật: $nodes[3].parent = 0, g = 1.0$. Tọa độ $N_3(1, 0)$.

$$h(3) = |1 - 1| + |0 - 2| = 0 + 2 = 2.0 \implies f(3) = 1.0 + 2.0 = 3.0$$

Đưa vào tập mở: $openSet = \{3\}$, gán $inOpenSet[3] = true$.

* **Xét neighbor = 4:** $tentative_g = 0.0 + weightMatrix[0][4] = 0.0 + 1.5 = 1.5 < nodes[4].g$.

Cập nhật: $nodes[4].parent = 0, g = 1.5$. Tọa độ $N_4(1, 1)$.

$$h(4) = |1 - 1| + |1 - 2| = 0 + 1 = 1.0 \implies f(4) = 1.5 + 1.0 = 2.5$$

Đưa vào tập mở: $openSet = \{3, 4\}$, gán $inOpenSet[4] = true$.

– **Trạng thái openSet cuối vòng:** $\{3(f = 3.0), 4(f = 2.5)\}$.

• **Bước 2: Vòng lặp thứ hai (Xét duyệt đỉnh 4)**

– Tìm nút f nhỏ nhất trong $openSet$: $nodes[4].f(2.5) < nodes[3].f(3.0) \implies curr = 4$.

– Xóa 4 khỏi $openSet$, gán $inOpenSet[4] = false, closedSet[4] = true$.

– Duyệt các lân cận chưa đóng của hàng 4 (đỉnh 3, và 5):

* **Xét neighbor = 2:** $tentative_g = 1.5 + weightMatrix[4][2] = 1.5 + 1.5 = 3.0 < nodes[2].g$.

Cập nhật: $nodes[2].parent = 4, g = 3.0$. Tọa độ $N_2(0, 2)$.

$$h(2) = |0 - 1| + |2 - 2| = 1 + 0 = 1.0 \implies f(2) = 3.0 + 1.0 = 4.0$$

Đưa vào tập mở: $openSet = \{3, 2\}$, gán $inOpenSet[2] = true$.

* **Xét neighbor = 3:** $tentative_g = 1.5 + weightMatrix[4][3] = 1.5 + 1.0 = 2.5$.

Vì $tentative_g(2.5) > nodes[3].g(1.0)$ nên bỏ qua (đường đi cũ tốt hơn).

* **Xét neighbor = 5:** $tentative_g = 1.5 + weightMatrix[4][5] = 1.5 + 1.0 = 2.5 < nodes[5].g$.

Cập nhật: $nodes[5].parent = 4, g = 2.5$. Tọa độ $N_5(1, 2)$.

$$h(5) = |1 - 1| + |2 - 2| = 0 \implies f(5) = 2.5 + 0.0 = 2.5$$

Đưa vào tập mở: $openSet = \{3, 2, 5\}$, gán $inOpenSet[5] = true$.

– **Trạng thái openSet cuối vòng:** $\{3(f = 3.0), 2(f = 4.0), 5(f = 2.5)\}$.

• **Bước 3: Vòng lặp thứ ba (Trích xuất đỉnh đích)**

- Đỉnh f nhỏ nhất là 5 ($nodes[5].f = 2.5$) $\implies curr = 5$. Xóa 5 khỏi openSet, $inOpenSet[5] = false$.
- Kiểm tra điều kiện: $curr == exitNode(5 == 5) \implies$ **Bật found = true**, lệnh break kích hoạt.

Kết quả xuất ra danh sách đơn PathNode (Mode 1):

Truy vết ngược $parent: 5 \rightarrow 4 \rightarrow 0$. Đảo chuỗi thu được: $\{0, 4, 5\}$. Định dạng chuỗi văn bản name:

Head $\rightarrow (0, 0) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow nullptr$

TRƯỜNG HỢP 2: MODE 2 (CHEBYSHEV DISTANCE)

• **Bước 0: Khởi tạo dữ liệu hệ thống**

- Khởi tạo mảng **nodes**: tất cả đều có $g = 1e9, f = 1e9, parent = -1$.
- Trạng thái tập hợp: $openSet = \{0\}, inOpenSet[0] = true, closedSet = \{all\ false\}$.
- Tọa độ nút gốc $N_0(0, 0)$, nút đích $N_5(1, 2)$.
- Tính chi phí nút gốc: $nodes[0].g = 0.0$.

$$nodes[0].h = \max(|0 - 1|, |0 - 2|) = \max(1, 2) = 2.0 \quad (\text{Chebyshev})$$

$$nodes[0].f = 0.0 + 2.0 = 2.0$$

• **Bước 1: Vòng lặp thứ nhất (Xét duyệt đỉnh 0)**

- Chọn $curr = 0$ ($f = 2.0$). Xóa 0, gán $inOpenSet[0] = false, closedSet[0] = true$.
- Duyệt các đỉnh lân cận hợp lệ (đỉnh 3 và 4):

* **Xét neighbor = 3:** $tentative_g = 0.0 + 1.0 = 1.0$.

Cập nhật: $nodes[3].parent = 0, g = 1.0$. Tọa độ $N_3(1, 0)$.

$$h(3) = \max(|1 - 1|, |0 - 2|) = \max(0, 2) = 2.0 \implies f(3) = 1.0 + 2.0 = 3.0$$

Thêm vào tập mở: $openSet = \{3\}$, gán $inOpenSet[3] = true$.

* **Xét neighbor = 4:** $tentative_g = 0.0 + 1.5 = 1.5$.

Cập nhật: $nodes[4].parent = 0, g = 1.5$. Tọa độ $N_4(1, 1)$.

$$h(4) = \max(|1 - 1|, |1 - 2|) = \max(0, 1) = 1.0 \implies f(4) = 1.5 + 1.0 = 2.5$$

Thêm vào tập mở: $openSet = \{3, 4\}$, gán $inOpenSet[4] = true$.

– **Trạng thái openSet cuối vòng:** $\{3(f = 3.0), 4(f = 2.5)\}$.

• **Bước 2: Vòng lặp thứ hai (Xét duyệt đỉnh 4)**

– Chọn nút f bé nhất: $nodes[4].f(2.5) < nodes[3].f(3.0) \implies curr = 4$.

– Xóa 4, gán $inOpenSet[4] = false$, $closedSet[4] = true$.

– Duyệt các lân cận chưa đóng của hàng 4 (đỉnh 2, 3, và 5):

* **Xét neighbor = 2:** $tentative_g = 1.5 + 1.5 = 3.0$.

Cập nhật: $nodes[2].parent = 4, g = 3.0$. Tọa độ $N_2(0, 2)$.

$$h(2) = \max(|0 - 1|, |2 - 2|) = \max(1, 0) = 1.0 \implies f(2) = 3.0 + 1.0 = 4.0$$

Thêm vào tập mở: $openSet = \{3, 2\}$, gán $inOpenSet[2] = true$.

* **Xét neighbor = 3:** $tentative_g = 1.5 + 1.0 = 2.5 > nodes[3].g(1.0) \rightarrow$
Bỏ qua.

* **Xét neighbor = 5:** $tentative_g = 1.5 + 1.0 = 2.5$.

Cập nhật: $nodes[5].parent = 4, g = 2.5$. Tọa độ $N_5(1, 2)$.

$$h(5) = \max(|1 - 1|, |2 - 2|) = 0 \implies f(5) = 2.5 + 0.0 = 2.5$$

Thêm vào tập mở: $openSet = \{3, 2, 5\}$, gán $inOpenSet[5] = true$.

– **Trạng thái openSet cuối vòng:** $\{3(f = 3.0), 2(f = 4.0), 5(f = 2.5)\}$.

• **Bước 3: Vòng lặp thứ ba (Trích xuất đỉnh đích)**

– Đỉnh f nhỏ nhất là 5 ($nodes[5].f = 2.5$) $\implies curr = 5$. Xóa 5, $inOpenSet[5] = false$.

– Kiểm tra điều kiện: $5 == 5 \implies$ **Bật found = true và dừng vòng lặp.**

Kết quả xuất ra danh sách đơn PathNode (Mode 2):

Truy vết ngược thu được chuỗi ID xuôi: $\{0, 4, 5\}$. Định dạng chuỗi văn bản **name**:

Head $\rightarrow (0, 0) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow nullptr$

4.3 Bảng tổng hợp so sánh hiệu suất định lượng của hai chế độ

Dựa trên kết quả mô phỏng chạy tay (Manual Trace) từ kịch bản lưới 2×3 với nút bẫy vật cản tại vị trí $N_1(0, 1)$, cấu trúc hiệu suất tìm đường của thuật toán A^* giữa hai chế độ Heuristic được tổng hợp chi tiết trong bảng dưới đây:

Tiêu chí so sánh	Mode 1 (Manhattan)	Mode 2 (Chebyshev)	Bản chất toán học / Hệ quả
Công thức khoảng cách	$ x_1 - x_2 + y_1 - y_2 $	$\max(x_1 - x_2 , y_1 - y_2)$	Mode 1 tính tổng các trục vuông góc; Mode 2 lấy giá trị đại số lớn nhất trên các trục.
Giá trị $h(0)$ tại Gốc $N_0(0, 0)$	3.0	2.0	Mode 1 cho giá trị ước lượng chặt và cao hơn trong kịch bản này.
Số vòng lặp Tìm kiếm (while)	3 vòng	3 vòng	Do kích thước lưới ví dụ nhỏ (2×3), cả hai chế độ đạt số vòng duyệt tương đương.
Số nút bị duyệt sai hướng	0 nút	0 nút	Cả hai chế độ đều hội tụ trực tiếp về chuỗi tối ưu $N_0 \rightarrow N_4 \rightarrow N_5$.
Tổng chi phí thực tế $g(5)$	2.5	2.5	Đều tìm ra chính xác lộ trình ngắn nhất đi qua góc chéo.
Độ mượt của lộ trình	Tốt	Tốt	Đều tận dụng được bước nhảy chéo trọng số 1.5.

Bảng 4.1: Bảng so sánh hiệu suất định lượng giữa Mode 1 và Mode 2

Phân tích chuyên sâu về mặt giải thuật toán học

1. Tính hợp lệ của hàm ước lượng (Heuristic Admissibility):

- Chi phí thực tế ngắn nhất tuyệt đối từ điểm xuất phát N_0 đến đích N_5 được ghi nhận là $h^*(0) = 2.5$.
- Tại trạng thái khởi tạo (Bước 0), Mode 1 cho ra $h(0) = 3.0$ (vi phạm điều kiện $h(n) \leq h^*(n)$ vì $3.0 > 2.5$). Điều này chứng minh trong không gian lưới cho

phép di chuyển 8 hướng (với chi phí đường chéo tối ưu chỉ bằng 1.5), **khoảng cách Manhattan không phải là một Heuristic hợp lệ (Admissible)**. Nó có xu hướng ước lượng “quá tay” chi phí cần thiết.

- Ngược lại, Mode 2 cho ra $h(0) = 2.0$ (thỏa mãn $2.0 \leq 2.5$). Do đó, **khoảng cách Chebyshev là một Heuristic hợp lệ (Admissible)** đối với không gian lưới ô vuông cho phép dịch chuyển 8 hướng, luôn bảo đảm tìm được lời giải tối ưu toàn cục.

2. Dự đoán hành vi giải thuật trên không gian bản đồ lớn ($m \times n \gg 100$):

- **Nếu cấu hình lưới chỉ cho phép 4 hướng (Ngang/Dọc):** Mode 1 (Manhattan) sẽ thể hiện hiệu suất vượt trội (tốc độ hội tụ nhanh, số nút mở rộng ít hơn) do hàm ước lượng trùng khớp hoàn toàn với quy luật hình học di chuyển thực tế.
- **Nếu cấu hình lưới cho phép 8 hướng:** Mode 2 (Chebyshev) sẽ hoạt động ổn định và chính xác hơn về mặt lý thuyết đồ thị. Trong không gian rộng lớn và trống trải, việc sử dụng Mode 1 (Manhattan) có nguy cơ đẩy giá trị chi phí ước lượng f lên quá cao ở một số hướng, khiến A^* có thể đưa ra lộ trình sơ tán bị lệch một chút so với đường đi ngắn nhất tuyệt đối, hoặc làm thuật toán mất nhiều thời gian xử lý hơn do hiện tượng không nhất quán (Inconsistency).